

correct thing to do, because the speeding up or slowing down of the sound can become noticeable.

The ideal solution is to apply a pitch shift without affecting the time axis. This can be done in a number of ways, including the *phase vocoder* and *time domain harmonic scaling* approaches. A complete description of these techniques is beyond our scope here, but you can read more about them at <http://www.dspdimension.com/admin/time-pitch-overview>.

Time-independent pitch shifting technology is an extremely powerful thing to have in your audio engine, in part because it also allows you to perform frequency-independent time scaling. So not only can you alter the pitch of sounds without changing timing for Doppler, you can also speed up or slow down sounds without altering their pitch for all sorts of other cool effects.

14.5 Audio Engine Architecture

To this point, we've discussed the concepts and methodologies behind 3D sound rendering, and the theory and technologies that underlie them. In this section, we'll turn our attention to the architecture of the software and hardware components used to implement a 3D audio rendering engine.

As with most computer systems, a game engine's audio rendering software is typically arranged into a "stack" of layered hardware and software components (see Figure 14.37).

- *Hardware* inevitably serves as the foundation of this structure, providing at minimum the necessary circuitry to drive the digital or analog speaker outputs that connect our PC or game console to a pair of headphones, a TV or a surround sound home theater system. Audio hardware may also provide "acceleration" to the software above it in the stack by supplying codecs, mixers, reverb tanks, effects units, waveform synthesizers and/or DSP chips in silicon. This hardware is often called the *sound card*

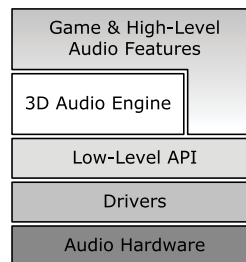


Figure 14.37. The audio hardware/software "stack."

because PCs sometimes provide their audio capabilities via a plug-in peripheral card.

- On a personal computer, the hardware is typically encapsulated in a *driver* layer, allowing the OS to support sound cards from a wide range of vendors.
- On both PCs and game consoles, the hardware and drivers are usually wrapped in a low-level *application programming interface* (API) designed to free the programmer from having to deal with the minutia of controlling the hardware and drivers directly.
- The *3D audio engine* itself is built on top of these foundations.

The feature set presented to the programmer by the audio hardware/software stack is usually modeled after the feature set of a *multi-channel mixer console* (http://en.wikipedia.org/wiki/Mixing_console) of the sort used in recording studios and at live concerts (see Figure 14.38). A mixer board can accept a relatively large number of audio inputs obtained from microphones and/or electronic instruments. The input sounds can be filtered and equalized, and reverb and other effects can be applied to them. The console is then used to mix all of the signals together, setting the relative volumes of the sounds as desired by the sound designer. The final mixed output is routed to the speakers (for a live performance) or to the individual channels of a multi-track recording.

In the same sense, the audio HW/SW stack must accept a large number of inputs (2D and 3D sounds), process them in various ways, mix them together so that their relative gains are set appropriately and finally pan these signals to the speaker output channels to produce the illusion of a three-dimensional soundscape for the human player.

14.5.1 The Audio Processing Pipeline

As we learned in Section 14.4.1, the process of rendering a 3D sound involves a number of discrete steps:

- For each 3D sound, a “dry” digital (PCM) signal must be *synthesized*.
- Distance-based attenuation is applied to provide a sense of distance from the listener, and *reverb* is applied to the signal to model the acoustics of the virtual listening space and to provide spatialization cues to the listener. This produces a new “wet” signal.
- The wet and dry signals are *panned* (independently) to one or more speakers in order to produce the final “image” of each signal in three-dimensional space.



Figure 14.38. A multi-channel mixer console by Focusrite with support for 72 inputs and 48 outputs.

- The panned multi-channel signals of all the 3D sounds are *mixed* together into a single multi-channel signal, which is either sent through a parallel bank of DACs and amps to drive the analog speaker outputs or sent directly to a digital output such as HDMI or S/PDIF.

Clearly, we think of the process of rendering 3D audio as a *pipeline*. And because a game world typically contains a large number of sound sources, multiple instances of this pipeline are in flight simultaneously. For this reason,

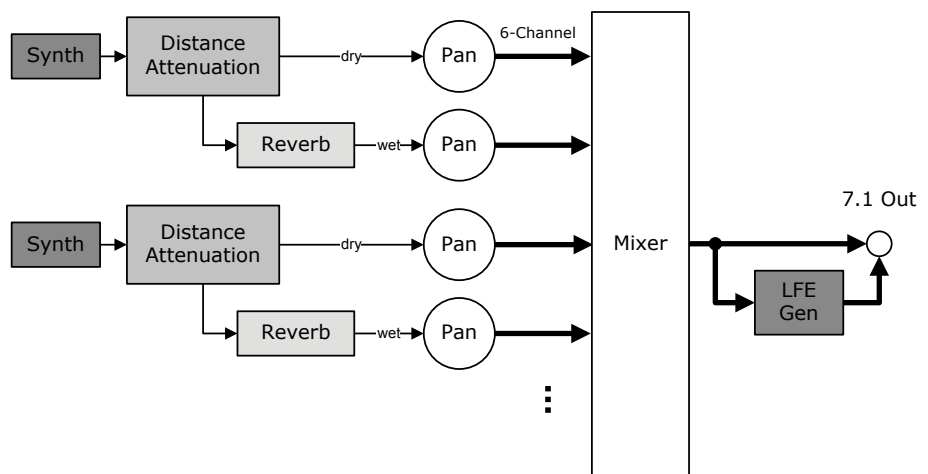


Figure 14.39. The audio processing graph (pipeline).

the audio processing pipeline is sometimes called the *audio processing graph*. It truly is a graph of interconnected components, ultimately culminating in the handful of speaker channels that comprise the final mixed, panned output. Figure 14.39 presents a high-level view of the audio graph.

14.5.2 Concepts and Terminology

Before we can explore the audio processing pipeline in any depth, we need to become familiar with a few concepts and the terminology used to describe them.

14.5.2.1 Voices

Each 2D or 3D sound passing through the audio rendering graph is called a *voice*. This term comes from the early days of electronic music: A synthesizer would produce musical notes via a set of waveform generators called “voices.”

A synthesizer contains a limited number of waveform generator circuits, so electronic musicians speak of how many simultaneous voices their synth can produce. In the same sense, a game’s audio rendering engine typically has a limited number of codecs, reverb units and so on. The maximum number of voices supported by a particular audio HW/SW stack is dictated by the number of independent parallel pathways through the audio graph. This number is generally bounded by limited memory resources, limited hardware resources and/or processing power limitations. This number is sometimes referred to as the degree of *polyphony* supported by the system.

2D Voices

A game’s audio rendering pipeline must also be capable of handling 2D sounds, such as music, menu sound effects, narrator voice-overs and so on. 2D voices are also processed by the audio rendering pipeline. The main things that differentiate 2D sound processing from 3D processing are:

- 2D sounds originate as multi-channel signals, one for each available speaker, whereas 3D sounds originate as dry monophonic signals. As such, 2D sounds do not pass through a pan pot.
- A 2D sound may contain “baked” reverb or other effects. If so, the sound may not make use of the reverb capabilities of the rendering engine.

As such, 2D sounds typically enter the pipeline just prior to the master mixer, where they are combined with the 3D sounds to produce the final “mix.”

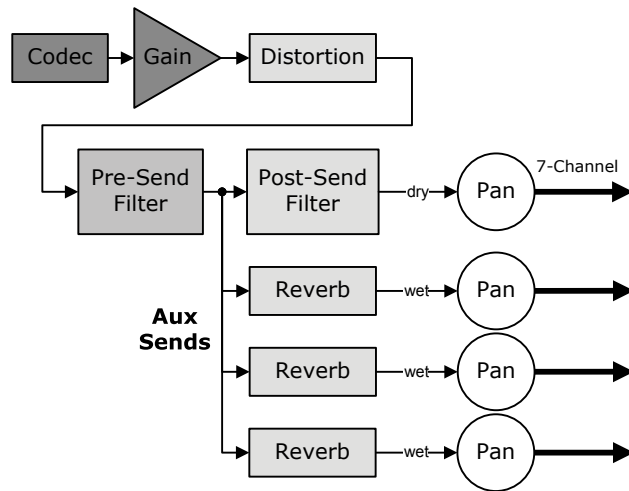


Figure 14.40. The pipeline through which an individual 3D voice passes on its way through the audio graph.

14.5.2.2 Buses

The interconnections between the components that make up the audio graph are called *buses*. In electronics, a bus is a circuit whose primary purpose is to connect other circuits to one another. In software, a bus is nothing more than a logical construct that describes the presence of an interconnection between components.

14.5.3 The Voice Bus

Figure 14.40 presents a more-detailed view of the pipeline of components through which a single 3D voice passes as it is rendered by the audio engine. In the following sections, we'll explore each of these components in detail and learn why they are interconnected in the way that they are.

14.5.3.1 Sound Synthesis: Codecs

An audio signal passes through the rendering graph in digital form. The term *synthesis* describes the process of generating these digital signals. Audio signals may be synthesized by simply “playing back” a pre-recorded audio clip. They might also be procedurally generated, perhaps by combining one or more fundamental waveforms (sinusoid, square wave, sawtooth, etc.), and/or by applying various filters to a harmonically rich noise signal. Since

most games use pre-recorded audio clips almost exclusively, we'll restrict our discussion to them here.

Pre-recorded audio clips can be provided to the game engine in any one of the myriad compressed and uncompressed audio file formats in use today (see Section 14.3.2.3). Raw PCM data is the “canonical” format accepted by the various components in the audio processing graph. Therefore, a device or software component known as a *codec* is used to convert each source audio clip into a raw PCM data stream. The codec interprets the source data format, decompresses the data if necessary, and then transmits it onto the voice bus for its journey through the audio processing graph.

14.5.3.2 Gain Control

The loudness of each source sound in the 3D world can be controlled in a number of ways: When recording the audio clip, we can set the recording levels to produce a sound at the desired loudness. We can process the clip in an offline tool to adjust its gain. At runtime, we can also dynamically adjust the volume of the clip using a *gain control* component within the audio graph. See Section 14.3.1.7 for a detailed discussion of gain control.

14.5.3.3 Aux Sends

When a sound engineer at a recording studio or live concert wants to apply effects to a sound, he or she can route the sound out of the multi-channel mixing console, through an effects “pedal,” and then back into the mixing board for further processing. These outputs are known as *auxiliary send* outputs, or *aux sends* for short.

Within the audio processing graph, the term “aux send” is used in an analogous manner: It describes a bifurcation point in the pipeline, splitting the signal into two parallel signals. One of these signals is for the dry component of the sound. The other is piped through a reverb/effects component to create the wet component of the sound.

14.5.3.4 Reverb

The wet signal path is typically routed through a component that adds early reflections and late reverberations. Reverb might be implemented using a convolution, as described in Section 14.4.5.1. If convolution is not practical in real time, either because the console or PC lacks DSP hardware or because the game's CPU and/or memory budgets are insufficient, reverb can be implemented using a *reverb tank*. This is essentially a buffering system that caches time-delayed copies of a sound that are then mixed with the original to mimic early reflections and/or late reverberations, combined with a *filter* to mimic the

interference effects and general attenuation of high-frequency components in the reflected sound waves.

14.5.3.5 Pre-Send Filter

The voice pipeline typically includes a filter that is applied before the aux send bifurcation, and therefore applies to both the dry and wet components of the sound. This is called a *pre-send filter*. It is generally used to model phenomena that occur at the source of the sound. For example, we could mimic the sound of someone wearing a gas mask with a pre-send filter.

14.5.3.6 Post-Send Filter

Another filter is typically provided after the aux send bifurcation. As such, this filter only applies to the dry component of the sound. This filter can be useful for modeling the muffling effect of an obstruction/occlusion on the direct sound path. At Naughty Dog, we also use a post-send filter to implement the frequency-specific fall-off that occurs due to atmospheric absorption (see Section 14.1.3.2).

14.5.3.7 Pan Pots

The dry and wet components of a 3D sound are monophonic signals throughout their journey along the voice bus. At the very end of the pipeline, each one of these two mono signals must be panned to the two stereo speakers/headphones or the five or seven surround sound speakers. For this reason, every 3D voice bus terminates in two or more pan pots, one for the dry signal and one or more for the wet. The components may be panned differently. The dry signal is panned according to the actual location of the source. The wet signal, however, may be panned with a wider focus to simulate the way in which reflected sound waves tend to impinge on the listener's head from various directions. If the sound is coming from a narrow doorway, the focus of the wet signal might be only a few degrees. But if the listener is standing in the center of a cavernous hall, the wet signal should probably be given a 360-degree focus (i.e., it should be rendered in all speakers equally).

14.5.4 Master Mixer

Each pan pot's output is a multi-channel bus, containing signals for each of the desired output channels (stereo or surround sound). A game typically has a large number of 3D sounds playing simultaneously. The *master mixer* takes all of these multi-channel inputs and mixes them together into a single multi-channel signal for output to the speakers.

Depending on the specifics of the implementation, the master mixer might be implemented in hardware, or it might live entirely in software. If the master mix is performed in hardware, the sound card designer has the option of performing an analog mix or a digital mix. (Software can only do digital mixing, for obvious reasons.)

14.5.4.1 Analog Mixing

An analog mixer is essentially just an summation circuit—the amplitudes of the individual input signals are added together, and the resultant wave’s amplitude is then attenuated to fall back within the desired signal voltage range.

14.5.4.2 Digital Mixing

Mixing can also be performed *digitally* by software running on a dedicated DSP chip or a general-purpose CPU. A digital mixer takes multiple PCM data streams as its inputs, and produces a single PCM data stream at its output.

A digital mixer’s job is a little more complicated than that of an analog mixer, because the collection of PCM channels it is combining may have been recorded at different sample rates and/or different bit depths. Two processes known as *sample depth conversion* and *sample rate conversion* must be executed on all of the mixer’s input signals to bring them into a common format. Once this has been done, mixing again becomes trivial. At each time index, the values of all the input samples are simply added together, and the final output amplitude is adjusted if necessary to bring the combined signal into the desired volume range.

14.5.4.3 Sample Depth Conversion

If the *bit depths* of the mixer’s input signals differ, *sample depth conversion* can be used to convert them to a common format. This operation is trivial. We simply de-quantize the input sample values into floating-point format, and then re-quantize each one at the desired output bit depth. See Section 12.8.2 for all the gory details on quantization.

14.5.4.4 Sample Rate Conversion

If the *sample rates* of the input signals differ, *sample rate conversion* must be used to convert them all into the desired output sample rate prior to mixing. In principle, this involves converting the signal into analog form, and then re-sampling it at the desired rate (which could be done using D/A and A/D hardware). In practice, analog sample rate conversion tends to introduce unwanted noise, so the conversion is almost always accomplished by running a direct digital-to-digital algorithm directly on the PCM data stream.

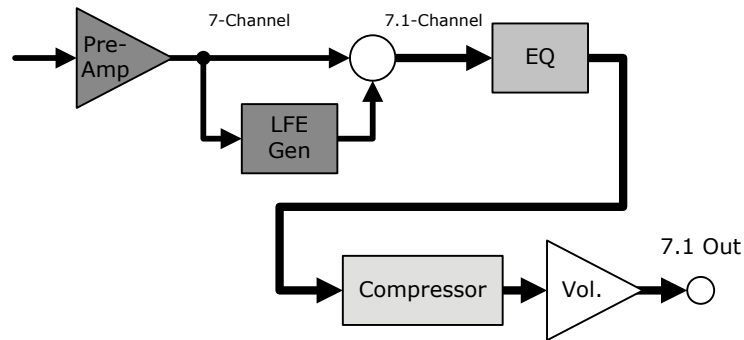


Figure 14.41. A typical master output bus.

An understanding of signal processing theory (see Section 14.2) is necessary to fully understand how these algorithms function, and a full discussion is beyond our scope here. But in certain simple cases, the concept is easy enough to grasp. For example, if we are *doubling* the sample rate, we can interpolate adjacent samples and insert these values as new samples, thereby doubling the number of samples. It's not quite as simple as this—one must take care to avoid introducing aliasing into the resulting signal, for example. See http://en.wikipedia.org/wiki/Sample_rate_conversion for a detailed discussion of sample rate conversion.

14.5.5 The Master Output Bus

Once the voices have been mixed, they are processed by the *master output bus*. This is a collection of components that process the output prior to sending it to the speakers. A typical master output bus is depicted in Figure 14.41, and its components are described briefly below. Every audio engine does things a bit differently, and not all engines support all of the components described below. Some engines may also introduce additional components not shown here.

- *Pre-amp.* The pre-amp allows the master signal's gain to be trimmed prior to passing through the remainder of the output bus.
- *LFE generator.* As we mentioned in Section 14.4.4, a pan pot only drives the two, five or seven “main” speakers of a stereo or surround sound system. The LFE (subwoofer) channel does not contribute to the positioning of a sound's 3D image. An *LFE generator* is a component that extracts the very lowest frequencies of the final mixed signal and uses this to drive the LFE channel.

- *Equalizer*. Most audio engines provide some kind of *equalizer* (EQ). As described in Section 14.2.5.8, an EQ allows specific frequency bands in the signal to be boosted or attenuated individually. A typical EQ divides the spectrum up into anywhere from four to tens of individually tunable bands.
- *Compressor*. A compressor performs *dynamic range compression* (DRC) on the audio signal. A compressor reduces the volume of the loudest portions of the signal and/or increases the volume of the quietest moments. It does this automatically by analyzing the input signal's volume characteristics and adjusting the compression dynamically. See http://en.wikipedia.org/wiki/Dynamic_range_compression for a detailed discussion of DRC.
- *Master gain control*. This component allows the overall volume of the entire game to be controlled.
- *Outputs*. The output of the master bus is a collection of line-level analog signals corresponding to the speaker channels and/or a digital HDMI or S/PDIF multi-channel signal, suitable for transmission to a TV or a home theater system.

14.5.6 Implementing a Bus

14.5.6.1 Analog Buses

An analog bus is implemented via a number of parallel electronic connections. To carry a monophonic audio signal, we need two parallel wires or “lines” on the circuit board: one to carry the voltage signal itself, and one to serve as ground.

An analog bus operates pretty much instantaneously. The output signal from an upstream component is immediately consumed by the next downstream component, because the signal is a continuous physical phenomenon. Such circuits are quite simple. The only real complication is ensuring that the voltage levels and impedances of the input and output signals match.

14.5.6.2 Digital Buses

One could imagine using simple digital circuitry to build instantaneous connections between our digital components. However, this would require the connected components to run in perfect lock-step: At the exact moment that the sender produces a byte of data, the receiver would have to consume it. Otherwise, the byte would be lost.

To overcome the synchronization problems inherent in connecting two digital components, *ring buffers* are typically used at the input and/or output of

each component. A ring buffer is a buffer that can be shared by two clients—one *reader* and one *writer*. To make this work, we maintain two pointers or indices within the buffer, called the *read head* and the *write head*. The reader consumes data at the read head, advancing it forward in the buffer as data is consumed, and wrapping when the end of the buffer is reached. The writer stores data into the buffer at the write head, advancing and wrapping as well. Neither head is permitted to “pass” the other, which guarantees that the two clients cannot conflict with one other (i.e., reading data that hasn’t been written yet, or writing over top of data that is currently being read).

The simplest way to connect, say, the digital output of a codec to the digital input of a DAC is to use a *shared* ring buffer. The codec writes to the exact same buffer read by the DAC.

While simple, the shared buffer approach only works when the two components have access to the same physical memory. This is trivial to do when the components are running in threads on a single CPU. To make a shared memory approach work between two separate operating system *processes*, each of which has its own private virtual memory space, the OS needs to provide a mechanism for mapping the same physical memory into each process’s virtual address space. This is usually only possible when the two processes are running on the same core, or on different cores within a multicore computer.

If the two components are running on different cores that cannot share memory (as would be the case if one were running on the PC and the other on a plug-in sound card, for example), then each component needs its own input or output buffer. Data must be *copied* from the output buffer of one component to the input buffer of the other. This might be accomplished via a *direct memory access controller* (DMAC), as is the case when transferring data between the PPU and the SPUs on the PS3. It might also be accomplished via a specialized bus, such as the ubiquitous PCI Express (PCIe) bus that is used to connect the main CPU core(s) to plug-in peripheral cards on a PC.

14.5.6.3 Bus Latency

In order to play sound, the game or application must feed audio data periodically into the codecs that ultimately drive the speaker outputs. We call this *servicing* the audio. The rate at which the game or app services its audio is crucial to proper sound production: If packets are sent too infrequently, the buffers will *underflow*, meaning that the device consumes all of the data before a new packet arrives. This causes the audio to drop out briefly while the software catches up. If packets are sent too often, the PCM buffers can *overflow*, causing packets to be lost. This causes the audio to seem to “skip.”

The size of the input and output buffers that comprise a digital bus dictates the *latency* of the sound system—in other words, how much delay is introduced by the bus. If the buffers are very small, latency is minimized, but this places a greater burden on the CPU because it must feed the buffers more frequently. Likewise, larger buffers translate into less CPU load but also higher latency. We usually measure the latency of a piece of audio hardware in milliseconds, rather than measuring the size of the buffers in bytes. This is done because buffer size depends upon the data format and the degree of compression supported by the codec, but the latency is really what we care about when trying to produce high-fidelity sound.

How much latency is acceptable? This depends on the application. Professional audio systems require very short latencies—on the order of 0.5 ms. This is because audio signals are often fed through a network of audio hardware before being synchronized with each other and often to a video signal as well. Every time latency is introduced by the hardware, accurate synchronization becomes more difficult.

Game consoles, on the other hand, can tolerate a longer latency. In a game, all we care about is synchronizing the audio and the graphics. If the game is rendering at 60 FPS, this translates into $1/60 = 16.6$ ms per frame. As long as the audio isn't delayed by more than 16 ms, we know it will be in sync with graphics rendered for that same frame. (In fact, many games use double or triple buffering for their rendering engines, which introduces one or two frames of delay between the time the game requests that a frame be drawn and when that frame will actually appear on the TV screen. The television may also introduce a delay. As such, a triple-buffered 60 Hz game can actually tolerate an audio latency of $3 \times 16 = 48$ ms or more.) The PlayStation 3's DMA controller runs every 5.5 ms, so PS3 audio systems are typically configured such that the audio buffers can hold an integer multiple of 5.5 ms worth of audio.

14.5.7 Asset Management

14.5.7.1 Audio Clips

The most atomic audio asset is a *clip*—a single digital sound asset with its own local timeline (analogous to an animation clip). A clip is sometimes called a *sound buffer*, because the digital sample data is stored in a buffer. A clip might encapsulate monophonic audio data (typical for 3D sound assets), or it might contain multi-channel audio (typically used for 2D assets or stereo sound sources in 3D). A clip may be stored in any of the audio file formats supported by your engine.

14.5.7.2 Sound Cues

A *sound cue* is a collection of audio clips plus metadata that describes how they should be processed and played. Cues are usually the primary means by which the game can request sounds to be played. (Playing individual clips may or may not be supported by the engine.) Cues also serve as a convenient division-of-labor mechanism: The sound designers can craft the cues using an offline tool, without having to worry about how or when they'll be played in-game. And the game programmers can play the cues conveniently in response to relevant events in the game, without having to worry about micromanaging the details of playback.

There are many ways in which the collection of clips in a cue could be interpreted and played back. A cue might contain clips representing the six channels of a pre-mixed 5.1 music recording. A cue might also collect up a set of raw sounds, from which a random selection can be made, for the sake of variety. A cue might also be set up to play its collection of raw sounds in a predefined sequence. A cue typically specifies whether the sound(s) it encapsulates represent a one-shot sound or a looping sound.

Some audio engines permit a cue to provide one or more optional sound clips that only play if the main sound is *interrupted* midway through playing. For example, a vocal cue might include a “glottal stop” sound that is played only if the person’s line of dialog is interrupted. This feature can also be used to provide a distinct “tail” sound that is played when a looping cue is stopped. For example, a looping machine gun sound cue might use this “tail” clip feature to produce a suitable echoing fall-off sound when the firing ceases.

A cue’s metadata might include whether it is intended to be played in 3D or 2D; the FO min, FO max and fall-off curve of the sound source; group membership (see Section 14.5.8.1); and possibly any special effects, filtering or equalization that should be applied when the sound is played. In Sony’s *Scream* engine—the sound engine used by Naughty Dog in its *Uncharted* and *The Last of Us* series—a cue can contain arbitrary script code that allows a sound designer to completely control how the encapsulated sound asset(s) are played when the cue plays.

Playing a Cue

Every audio engine that supports the concept of cues provides an API for playing them. This API is usually the primary way—and sometimes the *only* way—for the game code to request that a sound be played.

The cue playing API generally allows the programmer to specify whether the cue should be played as a 2D or 3D sound, to provide 3D position and

velocity parameters, to specify whether the sound should loop or play only once, and to specify whether the source buffer is in-memory or streamed. The API usually also allows us to control the volume of the sound and possibly other aspects of playback.

Most APIs return a *sound handle* to the caller. This handle allows the program to keep track of a sound as it is playing, so that it can be modified or canceled before the sound ends. A sound handle is usually implemented under the hood as an index into a global handle table, rather than as a raw pointer to the data that describes the sound instance. That way, if the sound ends naturally, the handle can be “nulled out” automatically. A handle mechanism can also be used to make the system thread-safe—if one thread kills the sound, other threads that have handles to the sound will automatically see their handles become invalid.

14.5.7.3 Sound Banks

A 3D audio engine manages a *lot* of assets. The game world contains a large number of objects. Each object can generate a variety of sounds. And in addition to 3D sound effects, we have music, speech, menu sound effects and so on.

All of this audio data takes up an immense amount of space, so we can’t keep it all in memory at once. On the other hand, the individual audio clips are too fine-grained and too numerous for them to be managed on an individual basis. As such, most game engines package their sound clips and cues into coarse units called *sound banks*.

Some sound banks are loaded when the game starts up and left in memory forever. For example, the collection of sounds made by the player character are always needed, so we could keep them in memory indefinitely. Other banks might be loaded and unloaded dynamically as the needs of the game change. For example, the sounds in level A might not be used in level B, so we can load the “A” bank only when level A is being played. For example, in Naughty Dog’s *The Last of Us*, the sounds of rain, flowing water and the creaking of beams on the verge of collapse were only loaded when the player was in the tilted building in Boston.

Some sound engines permit banks to be *relocated* in memory. This feature can entirely eliminate the memory fragmentation problems that would otherwise arise as lots of differently sized banks are loaded and unloaded during gameplay. See Section 6.2.2.2 for more information on memory relocation.

14.5.7.4 Streaming Sounds

Some sounds are so long in duration that they cannot be conveniently stored in memory all at once. Music and speech are common examples. For these kinds of sounds, many game engines support *streaming audio*.

Streaming audio is possible because when playing a sound, the only information we actually need is the signal data at and around the current time index. To implement streaming, we maintain a relatively small ring buffer for each streaming sound. Prior to playing the sound, we pre-load a small chunk of it into the buffer, then play the sound normally. The audio pipeline consumes the data from the ring buffer as it plays, making room for us to load more data. As long as we keep filling the buffer with data before it is all consumed, our sound will play seamlessly.

14.5.8 Mixing Your Game

If we were to play every sound coming from every game object, properly attenuated and spatialized and acoustically modeled, using all the techniques and technology we've discussed thus far, what would be the result? We might expect the answer to this question to be "an incredibly immersive and believable soundscape that wins awards and makes us rich!" But what we'd actually get is cacophony.

What separates a good game from a great game is the *mix*—what you hear, how much of it you hear, and just as importantly what you don't hear. The goal of a game's sound designer is to produce a final mix that:

- sounds realistic and emersive;
- isn't too distracting, annoying or difficult to listen to;
- conveys all information relevant to gameplay and/or story effectively; and
- maintains a mood and tonal color that is always appropriate, given the events taking place in the game and the overall design of the game.

All sorts of different kinds of sounds must come together in the game's mix. These include music, speech, ambient sounds like rain, wind, insects or the creak of an old building, sound effects such as weapons fire, explosions and vehicles, and the bumps, slides and rolling sounds made by physically simulated objects.

Various techniques are employed to ensure the mix of the game meets these goals. We'll explore a few of them in the following sections.

14.5.8.1 Groups

The most obvious thing we can do to improve the mix of our game is to set the levels of all the source sounds in the 3D world appropriately. The important thing here is to ensure that each sound's gain is appropriate *relative* to the other sounds in the game. For example, footsteps should be quieter than gunfire.

In some games, the loudness of certain sounds needs to change dynamically. Often we want to control an entire category of sounds at once. For example, during a frenetic fight sequence, we might want to bring up the levels of the music and the weapons, and drop the volume of ancillary sound effects. Or during a quiet moment with characters talking to one another, we might want to boost the speech a little and tone down ambient sounds to ensure the dialog can be heard.

For this reason, many audio engines support the concept of *groups*—a concept “stolen” from our old friend the multi-channel mixing console. On a mixing board, a collection of sound inputs can be routed to an intermediate mixer circuit, combining them into a single “group signal.” The gain of this signal can then be controlled with a single knob on the board, thereby allowing the sound engineer to control the loudness of all input signals at once.

In the software world, groups are implemented by simply categorizing sound cues, rather than physically mixing their signals together. For example, we can classify a cue as being music, a sound effect, a weapon, a line of speech and so on. Then, the engine can provide a means of controlling the gains of all sounds in each category with a single control value. Groups usually also allow entire categories of sound to be paused, restarted and muted conveniently with a simple API call.

Some sound engines do also provide a mechanism for physically mixing groups of audio signals into a single signal, just as is done when working with groups on a mixing console. In Sony's *Scream* engine, this is called generating a *pre-master submix*. After the relative gains of the signals in the group have been locked down by the submix, the resulting signal can be routed through additional filters or other processing stages. This gives the sound designer even more control over the mix of the game.

14.5.8.2 Ducking

Ducking is a technique in which the volume/gain of certain sounds are temporarily reduced in order to make other sounds more audible. For example, when a character is speaking, the level of background noise could be reduced automatically to make the dialog more audible.

Ducking can be triggered in numerous ways. The presence of one particular type of sound might be used to duck another category of sounds. A game event might trigger a duck programmatically. Any triggering mechanism that is deemed appropriate can be used to initiate a duck.

The reduction of volume caused by a duck is typically accomplished via the group categorization system: When one category of sound is playing, it can automatically duck one or more other categories by various amounts. Or the game code can call a function to duck a group of sounds programmatically.

Ducking can also be performed by routing one sound signal into the *side-chain* input of the dynamic range compressor (DRC) on a different voice's bus. Recall from Section 14.5.5 that a DRC analyzes the volume characteristics of a signal and automatically compresses the loudness of the signal appropriately. When a side-chain input is connected to a DRC, it analyzes the *side-chain* signal when deciding how to adjust the volume. So, we can arrange for increased loudness in one signal to cause a decrease in the dynamic range of another signal.

14.5.8.3 Bus Presets and Mix Snapshots

Many sound engines allow the sound designer to set up configuration parameters, save them off and then recall and apply them conveniently at runtime. In Sony's *Scream* engine, these come in two basic flavors: *bus presets* and *mix snapshots*.

A bus preset is a set of configuration parameters that control aspects of the components on a single bus (voice bus or master output bus). For example, a bus preset might describe one particular reverb set-up that mimics, say, the acoustics of a large open hall, or the interior of a car, or a small broom closet. Or a bus preset might control the DRC settings on the master output bus. Many such presets can be created by the sound designer, and the appropriate ones activated at runtime as the game requires.

A mix snapshot is the same kind of idea applied to gain control. The gains of the various channels within a group can be established a priori and then applied at runtime as needed.

14.5.8.4 Instance Limiting

Instance limiting is a means of controlling the number of sounds that are permitted to play simultaneously. For example, even though 20 NPCs are all firing their guns at once, we might only play the three or four gun sounds that are nearest to the listener. Instance limiting is important for two reasons: First, it's a great way to prevent cacophony. Second, a sound engine typically supports only a fixed number of simultaneous voices, either because of hardware

limitations (e.g., the sound card only has N codecs) or because of memory or processor bandwidth limitations in the software, so we must use them wisely.

Per-Group Limits

Instance limiting is sometimes applied differently to different groups of sounds. For example, we might specify that we should play up to four guns simultaneously, hear up to three people speaking at a time and allow up to five other sound effects to play at once, plus up to two overlapping music tracks.

Prioritization and Voice Stealing

In a 3D game with lots of dynamic elements, there may be more sounds playing at any given time than the system has voices for. Some sound engines support a large number (or even an infinite number) of *virtual voices*. Each virtual voice represents a sound that is technically playing, but that can be muted or stopped temporarily so it ceases to occupy valuable hardware or software resources. The engine uses various criteria to dynamically determine which virtual voices should be mapped to “real” voices at any given moment.

One of the simplest ways to limit the number of sounds playing simultaneously is to assign a maximum radius to every 3D sound source. As we saw in Section 14.4.3.1, this is the FO max radius. If the listener is beyond this distance from the sound, it is considered inaudible and its virtual voice is temporarily muted or stopped, freeing its resources for use by other voices. The process of automatically silencing virtual voices is called *voice stealing*.

Another common approach is to assign each cue or group of cues a *priority*. When too many virtual voices are playing at once, those with lower priorities can be silenced (stolen) in favor of higher priority voices.

Sound engines may also provide various other mechanisms for controlling the details of the voice stealing algorithm. For example, a cue might be given a minimum play time, after which its voice is permitted to be stolen. Sounds might be faded out rather than cut off abruptly when their voice is stolen. And some cues might be temporarily marked as “unstealable” to ensure that they play, even despite their priority settings.

14.5.8.5 Mixing In-Game Cinematics

Under normal gameplay conditions, the listener or “virtual microphone” is typically positioned at or near the location of the camera, and the sound sources are modeled where they really are in the environment. Distance-based attenuation, direct and indirect sound path determination, voice limiting—all

are determined using these realistic positions.

However, during an *in-game cinematic*—a portion of the game in which player control is suspended so that a story moment can take place—the camera often pans out away from the player’s head. This kind of thing tends to wreak havoc with our 3D audio system. We could just keep the listener/mic locked to the location of the camera; but this is not always appropriate. For example, if there’s a long shot of two characters speaking, we probably still want to mix so that the characters’ voices can be heard, even though physically speaking they are too far away to be heard. In this case, we might want to detach the listener from the camera, and artificially position it nearer to the characters.

Mixing in-game cinematics is a lot closer to mixing a film. As such, a sound engine needs to be capable of “breaking the rules” and doing things that aren’t necessarily physically realistic.

14.5.9 Audio Engine Survey

It should be evident by now that creating a 3D audio engine is a massive undertaking. Luckily for us, lots of people have already put a great deal of effort into this task, and the result is a wide range of audio software that we can use pretty much out of the box. This ranges from low-level sound libraries all the way to fully featured 3D audio rendering engines.

In the following sections, we’ll survey a few of the most common audio libraries and engines. Some of these are specific to a particular target platform, while others are cross-platform.

14.5.9.1 Windows: The Universal Audio Architecture

In the early days of PC gaming, the feature set and architecture of PC sound cards varied a great deal from platform to platform and vendor to vendor. Microsoft attempted to encapsulate all of this diversity within its DirectSound API, supported by the Windows Driver Model (WDM) and the Kernel Audio Mixer (KMixer) driver. However, because vendors could not agree on a common feature set or set of standard interfaces, the same functionality would often be realized in very different ways on different sound cards. This required the operating system to manage a very large number of incompatible driver interfaces.

For Windows Vista and beyond, Microsoft introduced a new standard called the Universal Audio Architecture (UAA). Only a limited set of hardware features are supported by the standard UAA driver API—all remaining features are implemented in software (although hardware manufacturers

are still free to provide additional “hardware acceleration” features, as long as they provide custom drivers to expose them). Although the introduction of UAA limited the competitive advantage of prominent sound card vendors like Creative Labs, it did have the desired effect of creating a solid, feature-rich standard, which could be used by games and PC applications in a convenient way.

The UAA standard had another positive effect on the user’s aural experience. In the old DirectSound days, a game could take complete control of the sound card, meaning that sounds coming from the OS or other applications such as an email program would be “locked out” and their sounds would not play while the game was running. The new UAA architecture allowed the OS to claim ultimate control over the final mix heard through the PC’s speakers. Multiple applications could finally share the sound card in a reasonable and consistent way. Search online for “Universal Audio Architecture” to find more information on UAA.

The UAA is implemented on Windows by the so-called Windows Audio Session API, or WASAPI for short. This API is not really intended for use by games. It supports most advanced audio processing features in software only, with limited support for hardware acceleration. Instead, games usually make use of the XAudio2 API, which is described in the next section.

14.5.9.2 XAudio2

XAudio 2 is the high-powered low-level API that provides access to the audio hardware on Xbox 360, Xbox One and Windows. It replaces DirectAudio and provides access to a wide range of hardware-accelerated features including programmable DSP effects, submixing, support for a wide range of compressed and uncompressed audio formats, and multirate processing to lighten the load on the main CPU(s).

Atop the XAudio2 API sits a 3D audio rendering library called X3DAudio. These APIs are also available on the Windows platform for use by PC games. Microsoft used to offer a powerful audio authoring tool called the “cross-platform audio creation tool” or XACT for short, which was intended for use with XNA Game Studio, but neither XNA nor XACT are supported any longer.

14.5.9.3 Scream and BoomRangBuss

On the PS3 and PS4, Naughty Dog uses Sony’s 3D audio engine Scream and its synth library, BoomRangBuss.

The audio hardware on a PlayStation 3 is very much like a UAA-compliant audio device, supporting up to eight channels of audio for full 7.1 surround

sound support, plus a hardware mixer and HDMI, S/PDIF, analog and USB/Bluetooth outputs. This audio hardware is encapsulated by a collection of OS-level libraries including libaudio, libsynth and libmixer. On top of these libraries, game makers are free to implement their own audio software stacks. Sony also provides a powerful 3D-capable audio stack of its own called *Scream* which game studios can use “out of the box.” *Scream* is available on the PS3, PS4 and PSVita platforms. Its architecture mimics a fully featured multi-channel mixer console.

On top of *Scream*, Naughty Dog implemented a proprietary 3D environmental audio system for use on the *Uncharted* and *The Last of Us* series. This system provides stochastic obstruction/occlusion modeling and a portal-based audio rendering system that permits rendering a highly realistic soundscape.

Advanced Linux Sound Architecture

The Linux equivalent of the UAA driver model is called the Advanced Linux Sound Architecture (ALSA). This Linux kernel component replaced the original Open Sound System (OSSv3) as the standard way to expose audio functionality to applications and games. See http://www.alsa-project.org/main/index.php/Main_Page for more information on ALSA.

QNX Sound Architecture

QNX Sound Architecture (QSA) is a driver-level audio API for the QNX Neutrino real-time OS. As a game programmer, you’ll probably never use QNX. But its documentation does provide an excellent picture of the concepts and the typical feature set of audio hardware. See http://www.qnx.com/developers/docs/6.5.0/index.jsp?topic=%2Fcom.qnx.doc.neutrino_audio%2Fmixer.html for these docs.

14.5.9.4 Multiplatform 3D Audio Engines

A number of powerful, ready-to-use cross-platform 3D audio engines are available. We’ll outline the most well-known of these below.

- *OpenAL* is a cross-platform 3D audio rendering API that has been deliberately designed to mimic the design of the OpenGL graphics library. Early versions of the library were open source, but it is now licensed software. A number of vendors provide implementations of the OpenAL API spec, including OpenAL Soft (<http://kcat.strangesoft.net/openal.html>) and AeonWave-OpenAL (<http://www.adalin.com>).